

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3



Code of Conduct:

I _____Ganesh Karthick M / 192321138_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1.

Question:

A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer:

Testing should be performed as a planned engineering process instead of being carried out randomly. The banking team should establish a proper testing strategy based on requirements, risks, and business priorities.

Test cases should be designed systematically using techniques such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

An **RTM (Requirement Traceability Matrix)** can be used to ensure that every requirement has corresponding test cases. Tools such as Selenium can support automation, JMeter can be used for performance testing, and Jira can be used for defect tracking.

For example, in an online fund-transfer module, amounts such as ₹1, ₹10,000, ₹10,000.01, and invalid values can be tested. Different conditions such as insufficient balance, invalid account details, OTP failure, and transaction limits can also be verified.

Replacing ad-hoc testing with systematic, risk-based, automated, and documented testing improves coverage, identifies defects earlier, and reduces failures in production.

2.

Question:

An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Answer:

A structured testing process should be followed throughout the **Software Development Life Cycle (SDLC)**.

Requirement Analysis

The tester reviews requirements such as card validation, PIN verification, account balance, withdrawal limits, and cash availability.

Test Planning

A test plan defines the scope, resources, schedule, risks, environment, and tools required.

Test Case Design

Test cases are created using techniques such as equivalence partitioning, boundary value analysis, and decision tables.

Test Environment

The ATM simulator, database, card reader, and banking server are configured.

Test Execution

The tester executes the test cases and compares actual results with expected results.

Defect Management

Any failure is recorded with reproduction steps, expected and actual results, severity, and supporting evidence.

Retesting and Regression

Fixed defects are retested, while related existing functions are tested again to ensure that new changes have not caused additional problems.

For an ATM withdrawal, the tester should verify a valid card, correct PIN, sufficient balance, withdrawal amount, daily limit, cash dispensing, balance update, and receipt generation. Invalid PINs, insufficient funds, exceeded limits, and unavailable cash should also be tested.

Following the same testing process across all SDLC stages reduces missed scenarios and improves reliability, consistency, and overall system quality.

3.

Question:

A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer:

The terms **error**, **defect**, and **failure** represent different stages of a software problem.

An **error** is a human mistake made during requirements, design, coding, or testing. For example, a developer may misunderstand a requirement and assume that the transaction limit is ₹1,00,000 instead of ₹50,000.

A **defect** is the incorrect condition introduced into the software because of that error. In this example, the transaction validation code contains the wrong limit.

A **failure** occurs when the software executes the defective code and produces an incorrect result. If the system allows a customer to transfer ₹80,000 even though the limit is ₹50,000, it is a failure.

The relationship can be represented as:

Human Error → Software Defect → Execution → Failure

For login, incorrect password validation logic is a defect, while allowing an invalid password to successfully log in is a failure.

Poor understanding of these terms may result in vague reports such as *"Login is not working."* A proper defect report should contain the defect ID, module, steps to reproduce, expected result, actual result, severity, priority, environment, and screenshots or logs.

Clear terminology and structured reporting improve communication and make defect resolution faster.

4.

Question:

A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer:

Defect clustering means that a large percentage of defects are often found within a small number of modules. This principle can be used to allocate testing resources more effectively.

The team should study historical defect data and identify modules that frequently contain problems. These high-risk modules should receive more test cases and greater testing effort.

Other important testing principles include:

- Testing can reveal the presence of defects but cannot prove their complete absence.
- Exhaustive testing is generally impossible.
- Testing should begin as early as possible.
- Testing methods depend on the application context.

- Repeating identical tests may become less effective over time.

Test cases can be prioritized based on business importance, failure probability, previous defects, security impact, customer usage, and module complexity.

For example, if an e-commerce system has historically experienced most defects in payment and checkout, these modules should receive higher priority than less critical areas such as UI themes.

This approach helps testers concentrate on areas with greater risk, improves defect detection, reduces wasted effort, and makes testing more efficient.

5.

Question:

A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Answer:

A **defect repository** is a centralized collection of information about defects identified during software development and testing. It should be used not only for tracking bugs but also for preventing their recurrence.

Defects can originate during:

- Requirement analysis
- Design
- Coding
- Integration
- Configuration
- Testing

They can be classified into functional, performance, security, UI, database, integration, and compatibility defects. Severity and priority can also be assigned.

Historical repository data can reveal frequently defective modules and repeated defect types. For example, if payment calculations repeatedly contain errors, additional regression cases should be created for different payment conditions.

The repository can be analyzed for defect frequency, root causes, severity, affected modules, and release-wise trends.

To prevent repeated defects, the organization should perform root-cause analysis, update regression suites, conduct code reviews, improve requirement validation, and automate repeated tests.

If a tax calculation error appears in several releases, test cases covering different tax slabs should be permanently added to regression testing.

Therefore, a properly used defect repository acts as a knowledge base that helps reduce recurring defects and improves software quality.

6.

Question:

A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer:

Frequent crashes during fund transfers may be caused by problems in input validation, network communication, memory usage, concurrency, or backend integration.

Testing should begin with requirement analysis and risk identification. Important scenarios should include:

- Valid and invalid account numbers
- Different transfer amounts
- Insufficient balance
- Network interruption
- Server timeout
- Multiple simultaneous transfers
- Expired OTP
- Session timeout
- Different devices and operating systems

Crash logs and monitoring information should be analyzed to identify reproducible conditions. For example, if the application crashes whenever the network switches from Wi-Fi to mobile data, the tester can reproduce the condition and provide the logs to developers.

Useful testing techniques include boundary testing, equivalence partitioning, state transition testing, negative testing, compatibility testing, performance testing, and security testing.

Another important scenario is a transaction where money is deducted but the confirmation fails. The system should correctly handle the transaction without losing or duplicating the payment.

Automated regression tests should be executed whenever major changes are introduced. Systematic testing and continuous monitoring can significantly reduce future failures.

7.

Question:

A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer:

Testing and debugging are related activities, but they have different objectives.

Testing evaluates the software to identify defects and determine whether the application satisfies its requirements.

Debugging is the process of finding the root cause of an identified defect and correcting the software.

For example, if an ATM deducts ₹10,000 but displays ₹5,000, the tester identifies and reports the problem. The developer then investigates the calculation or database logic and fixes the underlying defect. After the fix, the tester performs retesting and regression testing.

Confusing testing with debugging can result in unclear responsibilities and inefficient defect resolution.

A suitable process is:

Requirement → Test Design → Test Execution → Defect Report → Debugging → Fix → Retesting → Regression Testing

Defect reports should contain reproducible steps, expected and actual results, logs, severity, and environment details.

Separating the two activities while maintaining proper collaboration helps detect defects systematically and ensures that they are resolved efficiently.

8.

Question:

An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer:

Cross-browser testing ensures that a web application functions correctly and provides a consistent user experience across different browsers, operating systems, and screen sizes.

Browser-related defects may occur because different browsers use different rendering engines and may handle JavaScript, CSS, and security rules differently.

Testing should cover commonly used browsers such as:

- Chrome
- Firefox
- Edge
- Safari

Different operating systems, devices, and screen resolutions should also be considered.

Important functions such as login, product search, cart, checkout, payment, and account management should be tested.

For example, if the Add to Cart button works in Chrome but not in Firefox, the tester should reproduce the problem and record the browser version, operating system, steps, screenshots, and logs.

Tools such as Selenium, Playwright, BrowserStack, LambdaTest, and browser developer tools can improve coverage.

Systematic cross-browser testing identifies compatibility problems earlier and helps ensure reliable behavior for users on different platforms.

9.

Question:

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer:

Incorrect salary calculations are mainly **functional or logical defects** because the implemented business rules do not produce the required results.

Possible categories include:

- Calculation or logic defects
- Requirement defects
- Database defects
- Integration defects
- Boundary defects
- Data validation defects

For example:

Basic Salary = ₹30,000

Allowance = ₹5,000

Deduction = ₹3,000

The expected salary is:

$$₹30,000 + ₹5,000 - ₹3,000 = ₹32,000$$

If the program calculates the salary incorrectly because of an incorrect formula, it represents a logical defect.

Testing should include equivalence partitioning, boundary value analysis, decision tables, data-driven testing, unit testing, integration testing, and regression testing.

Minimum and maximum salaries, zero allowances, maximum deductions, and tax limits should also be tested.

Code reviews, unit tests, static analysis, and automated regression testing can prevent similar problems in future releases.

Proper defect classification helps testers select suitable testing methods and improves payroll accuracy.

10.

Question:

A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer:

Random testing without proper planning can miss important requirements and edge conditions. Testing should therefore be treated as an organized engineering process.

A suitable testing flow is:

Requirement Analysis → Risk Assessment → Test Design → Test Data Preparation → Execution → Defect Reporting → Retesting → Regression Testing

A properly designed test case should contain:

- Test ID
- Requirement
- Preconditions
- Input data
- Steps
- Expected result
- Actual result
- Status

Important test design techniques include:

- **Equivalence Partitioning** – divides inputs into valid and invalid groups.

- **Boundary Value Analysis** – checks values near limits.
- **Decision Tables** – tests combinations of conditions.
- **State Transition Testing** – validates changes between system states.
- **Use Case Testing** – verifies complete user workflows.

For a password field accepting 8–16 characters, values such as 7, 8, 9, 15, 16, and 17 characters should be tested.

Random testing may test only an ordinary value and fail to identify boundary defects.

Structured test design improves coverage, reduces duplication, and makes testing repeatable and measurable.

11.

Question:

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer:

Data inconsistency occurs when different modules store or display different values for the same information. In a healthcare system, maintaining correct data is especially important.

Possible causes include:

- Incorrect database queries
- Data-mapping errors
- Synchronization problems
- API defects
- Validation errors
- Transaction failures
- Incorrect requirements

For example, if a patient's blood group is recorded as O+ in the registration module but appears as A+ in the medical-record module, the problem may be related to data mapping or synchronization.

The issue should be investigated using database testing, API testing, integration testing, data validation, end-to-end testing, and regression testing.

The same patient information should be traced across all relevant modules to ensure consistency. An RTM can also be used to connect requirements with test cases.

Defect reports should include the module, test data, reproduction steps, expected and actual values, relevant logs, severity, and environment.

Sensitive healthcare information should be protected by using controlled and anonymized test data.

Systematic database and integration testing helps maintain data integrity and consistency across modules.

12.

Question:

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Answer:

A defect repository should be used as a historical source of information for improving future testing, rather than simply as a list of reported bugs.

Each defect record should contain:

- Defect ID
- Module
- Defect type
- Root cause
- Severity
- Priority
- Release
- Status
- Resolution
- Related test case

For example:

Module	Defects
--------	---------

Payment	40
---------	----

Login	15
-------	----

Search	5
--------	---

This indicates that payment is a high-risk module and should receive greater testing attention.

If many payment defects are caused by validation problems, additional validation and boundary test cases should be introduced.

The organization should perform root-cause analysis, add production defects to regression suites, review requirements, conduct code reviews, increase testing of high-risk modules, and automate recurring scenarios.

If a tax calculation defect occurs repeatedly, a permanent automated regression test should be created for it.

Using historical defect information in this way helps identify patterns, improve test design, and prevent recurring problems.

13.

Question:

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer:

Failure during peak usage indicates insufficient performance and scalability testing. Functional testing alone cannot determine whether a system can support a large number of simultaneous users.

Testing gaps may include:

- Concurrent users
- Peak traffic
- Large database requests
- Server capacity
- Response time
- Network conditions
- Resource consumption
- Database bottlenecks

A performance test plan should define expected load, peak load, response-time requirements, test environment, performance metrics, and acceptance criteria.

Different performance testing methods should be applied:

- **Load Testing** – tests expected user traffic.
- **Stress Testing** – checks behavior beyond normal capacity.
- **Spike Testing** – evaluates sudden increases in users.
- **Endurance Testing** – checks long-duration operation.

For example, if 10,000 students are expected to access an online examination simultaneously, testing with only 100 users is inadequate.

Tools such as JMeter or LoadRunner can simulate large numbers of users while CPU, memory, database response time, and throughput are monitored.

Better performance planning helps identify bottlenecks before deployment and improves reliability during peak usage.

14.

Question:

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer:

Testers should participate from the beginning of the SDLC rather than joining only after development is completed. This approach is known as **shift-left testing**.

The tester can contribute during different stages:

- **Requirement Phase:** Identify ambiguity and check testability.
- **Design Phase:** Identify integration and usability risks.
- **Development Phase:** Review unit tests and participate in code reviews.
- **Testing Phase:** Perform functional, integration, system, and regression testing.
- **Deployment:** Verify release readiness.
- **Maintenance:** Perform regression and production validation.

For example, a banking requirement may state that users can transfer money within a specified limit. A tester can immediately ask about the minimum and maximum values, behavior when the limit is exceeded, network failures, and OTP requirements.

Finding such gaps before coding reduces rework and prevents requirement-related defects.

Regular requirement reviews, design discussions, Agile meetings, code reviews, and continuous testing should involve testers.

Early tester participation improves collaboration, reduces defects, and increases software quality.

15.

Question:

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer:

Incomplete test coverage means that only a portion of the possible inputs, conditions, requirements, or execution paths have been tested.

Testers should examine:

- Requirement coverage
- Functional coverage
- Code coverage
- Boundary conditions
- Negative scenarios
- User workflows
- Integration points

An **RTM** can be used to ensure that every requirement is linked to appropriate test cases.

For example, if a bank allows a maximum withdrawal of ₹20,000, testing only ₹5,000 is insufficient. Values such as ₹1, ₹19,999, ₹20,000, ₹20,001, negative amounts, and insufficient-balance conditions should be tested.

Suitable techniques include:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Pairwise Testing
- Risk-Based Testing

Automated regression testing can repeatedly execute a large number of cases.

Complete testing of every possible input is normally impractical, so representative, boundary, invalid, and high-risk scenarios should be selected systematically.

Improved coverage increases confidence and helps identify defects that ordinary test cases may miss.

16.

Question:

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer:

Poor defect reports make it difficult for developers to reproduce and understand problems. Therefore, defect reports should be complete, precise, and easy to reproduce.

A standard report should contain:

1. Defect ID

2. Title
3. Module
4. Environment
5. Preconditions
6. Steps to reproduce
7. Test data
8. Expected result
9. Actual result
10. Severity
11. Priority
12. Screenshots or logs
13. Reproducibility
14. Status

For example, instead of reporting *"Delivery tracking is not working,"* the tester should mention that the tracking status does not change after a delivery scan and provide the shipment ID, exact steps, expected status, actual status, timestamp, environment, and logs.

Severity represents the impact of the defect, while **priority** indicates the urgency of fixing it.

Using standardized templates and tools such as Jira improves communication and tracking.

Clear reports reduce investigation time, prevent duplicate issues, and help developers resolve defects more quickly.

17.

Question:

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer:

Process-based testing establishes a common and repeatable method for carrying out testing activities across different modules.

A standard testing lifecycle can be followed:

Requirement Analysis → Test Planning → Test Design → Environment Setup → Execution → Defect Management → Retesting → Regression Testing → Test Closure

Common organizational standards should be defined for:

- Test case formats
- Naming conventions
- Defect reporting

- Severity levels
- Entry and exit criteria
- Review procedures
- Automation practices

For example, if the login, payment, and shopping-cart teams use different test case formats, the organization can introduce one common template.

Each module should contain appropriate positive, negative, boundary, security, and regression tests.

Centralized test management tools, common checklists, periodic reviews, and process audits can ensure compliance.

Standardized testing makes the process repeatable and measurable, reduces missed scenarios, improves communication, and ensures consistent quality across modules.

18.

Question:

A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer:

Edge cases are often missed because testers focus mainly on normal input conditions. **Boundary Value Analysis (BVA)** is useful for identifying defects near the limits of accepted values.

Suppose a system accepts ages from **18 to 60**. Important test cases include:

Age	Expected Result
-----	-----------------

17	Invalid
----	---------

18	Valid
----	-------

19	Valid
----	-------

59	Valid
----	-------

60 Valid

61 Invalid

If the developer incorrectly uses a condition such as `age > 18`, users aged 18 may be rejected.

Edge cases can be missed due to incomplete requirements, poor test design, limited time, excessive focus on normal scenarios, or lack of historical defect analysis.

To improve testing, BVA should be combined with equivalence partitioning, decision tables, negative testing, and risk-based testing.

Minimum and maximum limits should be clearly defined in requirements. Any edge-case defect discovered in production should also be added to the regression suite.

This systematic approach improves the detection of boundary-related problems before deployment.

19.

Question:

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer:

A **defect repository** provides a centralized location for recording, tracking, and managing defects throughout their lifecycle. Proper documentation makes defect handling more organized and traceable.

A defect record should contain:

- Unique ID
- Title
- Description
- Module
- Environment
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Reporter
- Assignee

- Status
- Resolution
- Supporting evidence

For example, instead of simply stating "*Payment failed*," the tester should provide the transaction conditions, reproduction steps, expected and actual results, error message, and relevant logs.

A typical defect lifecycle is:

New → Assigned → Open → Fixed → Retest → Verified → Closed

If the issue remains after fixing:

Retest → Reopened

The repository can be analyzed to identify defects by module, severity, recurrence, root cause, and resolution time.

If database problems repeatedly occur in the payment module, additional database testing and code reviews can be introduced.

Standard templates, mandatory fields, clearly defined severity levels, and regular repository analysis improve defect management and help prevent repeated problems.

20.

Question:

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Answer:

Repeated transaction failures indicate that the current testing process is not sufficiently structured or risk-oriented. A redesigned strategy should cover requirements, risk assessment, systematic test design, integration, failure conditions, automation, and previous defects.

Requirement Analysis

The tester should identify rules related to valid accounts, available balance, transaction limits, OTP verification, duplicate transactions, and timeouts.

Risk-Based Planning

Critical operations such as fund transfers, payments, and balance updates should receive higher testing priority.

Test Design

The following techniques can be applied:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Negative Testing
- Use Case Testing

For a transaction limit of ₹50,000:

Input	Expected Result
₹1	Accepted
₹50,000	Accepted
₹50,001	Rejected
Negative amount	Rejected
Insufficient balance	Rejected

Integration Testing

The complete transaction flow should be tested:

Web/Mobile Application → API → Transaction Server → Database → Banking System

Failure Testing

Scenarios such as network failure, server timeout, duplicate requests, expired sessions, database failures, and expired OTPs should be included.

Automation and Regression

Automated regression tests should run whenever transaction-related code is changed.

Defect Repository



Historical transaction defects should be analyzed and converted into permanent regression test cases.

Combining risk-based testing, automation, boundary testing, integration testing, negative testing, and regression testing provides better defect detection, reduces recurring failures, and improves the reliability of banking transactions.